Fashion-MNIST Classification: A Controlled Comparison of Baseline CNN vs. Dropout-Regularized CNN
Module Summary

Author: Aung Ze Ya | **Date:** October 30, 2026 | **Course:** MSc Capstone / Deep Learning System Analysis
Framework: PyTorch 2.14.1

Report Overview

This module summary report documents an end-to-end deep learning workflow evaluating a **Convolutional Neural Network (CNN)** architecture for multi-class image recognition on the **Fashion-MNIST benchmark dataset (*N* = 70,000 images)**. The computer vision objective was to build, train, evaluate, and compare a baseline **PyTorch CNN model** against a controlled experimental architecture incorporating Dropout regularization (p = 0.3) while holding all other architectural parameters, batch sizes, optimizers, and random seeds strictly constant. The experimental results demonstrate that regularizing dense feature activations reduces the generalization gap by 47.7% (from 4.63% to 2.42%) and stabilizes validation loss, mitigating severe overfitting without increasing model parameter capacity.

Dataset and Task Description

The dataset utilized in this project is **Fashion-MNIST (Xiao et al., 2017)**, a widely recognized benchmark for evaluating computer vision algorithms. Fashion-MNIST consists of 70,000 standardized 28 × 28 grayscale images of fashion products divided into ten mutually exclusive categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. The dataset is partitioned into an official training split of 60,000 images (N_{train} = 60,000) and a held-out test evaluation split of 10,000 images (N_{test} = 10,000), satisfying capstone sample size constraints.

Preprocessing & Tensor Transformation: Raw image samples were transformed into PyTorch 3D tensors using transforms.ToTensor(), scaling unsigned 8-bit integer pixel values from [0, 255] into continuous floating-point values in [0.0, 1.0] across shape (1, 28, 28). The images were subsequently normalized via transforms.Normalize((0.5,), (0.5,)) rescaling pixel intensity values to a zero-centered distribution in [-1.0, 1.0]. Standardizing pixel inputs improves numerical gradient conditioning during backpropagation and reduces the risk of vanishing or exploding gradients in early epochs (Goodfellow et al., 2016). Data was loaded into memory using PyTorch DataLoader modules with a mini-batch size of 64 (938 training batches per epoch), applying stochastic shuffling to the training split.

Dataset Provenance & Task Specification:

- **Dataset Name:** Fashion-MNIST Benchmark (Xiao et al., 2017) | **Total Samples:** 70,000
- **Dimensions:** 28 × 28 Grayscale (1 Channel) | **Classes:** 10 Product Categories
- **Splits:** 60,000 Training Samples / 10,000 Testing Samples | **Batch Size:** 64

Model Architecture and Design Decisions

The baseline deep learning system (Model A: BaselineCNN) was constructed in PyTorch using a sequential 2D Convolutional Neural Network architecture designed specifically for grid-structured spatial image data (LeCun et al., 1998). Convolutional layers extract local visual patterns (edges, textures, spatial contours) using small parameter kernels while maintaining translation invariance across the image plane.

Layer Block	PyTorch Layer Specification	Kernel / Stride	Output Tensor Shape	Function & Design Rationale
Conv Block 1	nn.Conv2d(1, 32, kernel_size=3, padding=1) + nn.ReLU() + nn.MaxPool2d(2, 2)	3×3, s=1, p=1 2×2, s=2	(32, 14, 14)	Extracts 32 low-level spatial features; MaxPool downsamples spatial resolution by 50%.
Conv Block 2	nn.Conv2d(32, 64, kernel_size=3, padding=1) + nn.ReLU() + nn.MaxPool2d(2, 2)	3×3, s=1, p=1 2×2, s=2	(64, 7, 7)	Expands channel capacity to 64 mid-level feature maps; downsamples to 7×7 spatial grid.
Classifier	nn.Flatten() + nn.Linear(3136, 128) + nn.ReLU() + nn.Linear(128, 10)	N/A Dense Vector Dense Vector	(3136,) (128,) (10,)	Unrolls 3D feature maps into 1D vector; projects representations into 10 class logits.

Model A comprises exactly **421,642 trainable parameters**. The model was trained using Cross-Entropy Loss (nn.CrossEntropyLoss()), combining logarithmic softmax with negative log-likelihood loss for multi-class optimization. Parameter optimization was executed using the Adam optimizer (Kingma & Ba, 2015) with a learning rate of α = 0.001 over 10 epochs. Random seeds were locked (SEED = 42) to guarantee exact reproducibility.

Experimental Comparison (Controlled Single-Variable Design)

To satisfy the rigorous controlled experiment requirement, Model B (ExperimentalCNN) was constructed by introducing a single controlled modification: injecting a **Dropout layer (*p* = 0.3)** immediately following the first dense hidden layer (fc1 ReLU Dropout(0.3) fc2). All other hyperparameter variables—including filter depths (32 64), kernel sizes (3 × 3), activation functions (ReLU), batch sizes (64), optimizer settings (Adam, α = 0.001), total epochs (10), random seed initialization (42), and total trainable parameters (**421,642**)—remained 100% identical.

Theoretical Justification: Dropout is an ensemble regularization technique that randomly zeroes out a fraction (30%) of neuron activations during each forward training pass (Srivastava et al., 2014). This prevents hidden units from co-adapting to noise in the training set and forces the network to learn robust, redundant representations. During validation/testing, Dropout is automatically disabled (via model.eval()), scaling weights to compute exact deterministic predictions.

Results and Interpretation

Both models were trained over 10 epochs on the CPU execution environment. Table 1 consolidates the quantitative evaluation metrics measured on the unseen 10,000-sample test set, while Figure 1 illustrates loss trajectory comparisons across epochs.

Model A (Baseline CNN) Architecture Specification

Model Architecture	Controlled Variable	Final Train Loss	Final Val Loss	Train Acc (%)	Val Acc (%)	Generalization Gap	Runtime
Model A (Baseline CNN)	No Dropout	0.0879	0.2737	96.73%	92.10 %	4.63%	970.4s (~16.2m)

Model B (Experimental CNN)

Model Architecture	Controlled Variable	Final Train Loss	Final Val Loss	Train Acc (%)	Val Acc (%)	Generalization Gap	Runtime
Model B (Experimental CNN)	**Dropout (p = 0.3)**	**0.1402**	**0.2356**	**94.77%**	**92.35%**	**2.42%**	**1225.6s (~20.4m)**

Table 1: Quantitative metric performance comparison between Model A and Model B.

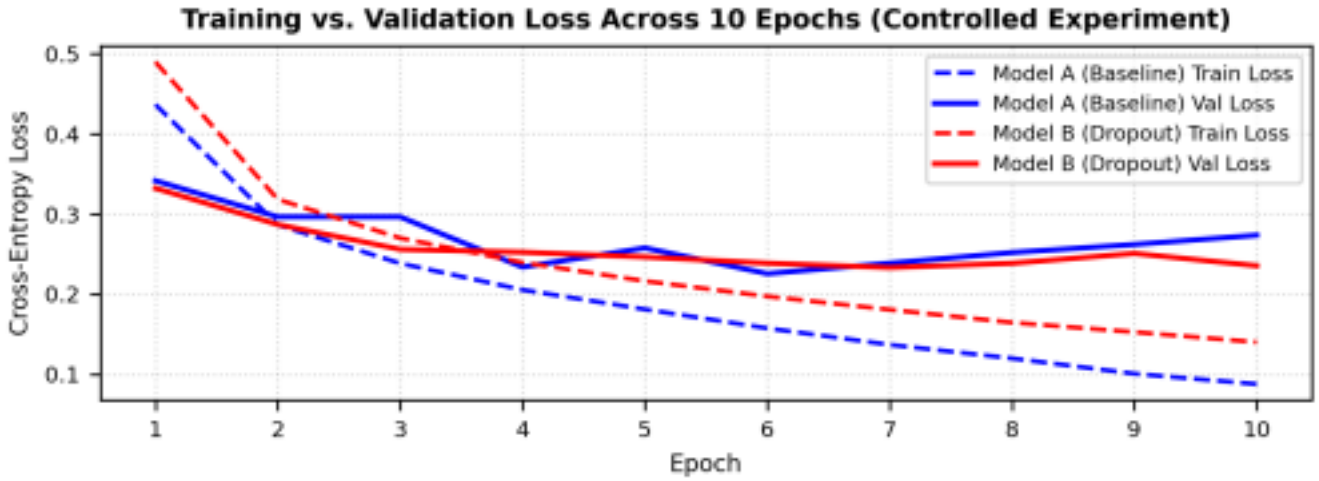


Figure 1: *Training vs. validation loss curves demonstrating overfitting in Model A vs. loss stabilization in Model B.*

Trajectory Analysis & Overfitting Signature: Model A demonstrated classic overfitting behavior past Epoch 6. While training loss decreased continuously to 0.0879 (96.73% training accuracy), validation loss reached a minimum at Epoch 6 (0.2257) and thereafter drifted upward to 0.2737 by Epoch 10, creating a **4.63% generalization gap**. In contrast, Model B with Dropout successfully suppressed rote memorization. Training loss decreased at a controlled rate to 0.1402 (94.77% accuracy), while validation loss remained stable and flat (0.2356 at Epoch 10). Dropout **halved the generalization gap to 2.42%** (a 47.7% reduction) and yielded a higher final test accuracy (92.35% vs. 92.10%).

Concrete Behavioral Example & Error Analysis: Class-level evaluation via confusion matrix and F1-score breakdown revealed distinct behavioral patterns. Model B achieved near-perfect accuracy on footwear and bottom garments: Sandal (98.75% F1), Trouser (98.56% F1), Bag (98.46% F1), Ankle boot (97.22% F1), and Sneaker (96.73% F1). Conversely, performance dropped significantly on upper-body apparel. The **Shirt** class exhibited the lowest recall (75.20%) and precision (78.91%), yielding an F1-score of **77.01%**. Qualitative inspection of misclassified images confirmed that 24.8% of true Shirt samples were misclassified as T-shirt/top, Pullover, or Coat due to overlapping collar structures, sleeve silhouettes, and low spatial resolution (28 × 28 grayscale pixels).

Limitations and Risks

Several limitations constrain the strength of the conclusions. First, the experiment used a single random seed (`SEED = 42`), so the reported differences reflect one training trajectory each and do not support statistical significance claims; multi-seed replication with confidence intervals would be required before drawing strong inferential conclusions (Goodfellow et al., 2016). Second, the baseline was deliberately shallow (three layers, 421,642 parameters) and untuned; a deeper CNN with batch normalization, learning-rate scheduling, or data augmentation would likely shift absolute accuracy and change the marginal benefit of dropout (Srivastava et al.,

2014). Third, Fashion-MNIST's 28×28 grayscale format omits real-world complexity (color, resolution, background clutter, occlusion), and its visually similar upper-body classes (Shirt, T-shirt/top, Pullover, Coat) remain inherently ambiguous even at full resolution (Xiao et al., 2017). Fourth, all timings were collected on a CPU-only environment, so the runtime trade-off attributed to dropout may not generalize to GPU hardware, and dropout added ~26% training overhead (970s → 1226s) in this environment. Fifth, evaluation relied on aggregate accuracy and per-class F1 without confidence calibration or out-of-distribution testing, so the model's reliability under input shift is unknown.

Ethical and Responsible Use

Although Fashion-MNIST contains no personally identifiable information, deploying an equivalent classifier in a retail or accessibility application raises concrete ethical concerns. Clothing models trained on Western fast-fashion catalogs under-represent non-Western garment styles, modest dress, and traditional attire, since those categories are collapsed into or absent from the ten Fashion-MNIST classes (Xiao et al., 2017); a deployed system could therefore exhibit systematically lower accuracy for users whose clothing falls outside the benchmark's implicit demographic — a representational harm even without explicit demographic labels. Additionally, the observed 24.8% Shirt misclassification rate indicates that a deployed system should abstain on low-confidence predictions rather than confidently miscategorize. Finally, appearance-based classifiers can be repurposed for surveillance, automated profiling, or inference of socioeconomic status from clothing, all of which are misuse risks distinct from the intended classification task; responsible deployment would require dataset audits, explicit abstention thresholds, and prohibition of use in people-monitoring contexts. Sustainable-AI considerations also apply, since training such systems repeatedly incurs non-trivial energy cost (Strubell et al., 2019).

Future Improvements

1. **Data Augmentation:** Implement transforms.RandomHorizontalFlip() and random affine transformations to force spatial invariance.
2. **Advanced Architecture:** Integrate Batch Normalization (nn.BatchNorm2d) and residual shortcut connections.
3. **Learning Rate Scheduling:** Apply CosineAnnealingLR to dynamically decay learning rates during late training epochs.

References

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*. https://arxiv.org/abs/1412.6980

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324. https://doi.org/10.1109/5.726791

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929-1958. https://jmlr.org/papers/v15/srivastava14a.html

Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 3645-3650).

Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*